

PROGRAM EXPLANATION

プログラム説明資料

Action3D

自作エンジンで動く 3D アクション

制作期間 2026年6月 -(開発中)
制作形態 個人開発
開発環境 C++ / DirectX 11 / HLSL

菊池 凌斗

Ryoto Kikuchi / HAL東京 ゲーム学科



作品概要と開発環境

— 作品概要

DirectX 11 の API を直接使い、描画・物理・アセット管理・オーディオまでを自作したゲームエンジンと、その上で動く3Dアクションゲーム。ゲーム側は、剣と「虚装」(体から伸びる影の武器)で敵の群れとボスと戦う。

— 開発環境

言語 / API	C++ / DirectX 11(HLSL)
ライブラリ	DirectXTex / cgltf / Dear ImGui / SDL3 / stb_truetype
音声	XAudio2 / X3DAudio / Media Foundation
期間・形態	2026/6 -(開発中)・個人開発
私の役割	設計・実装(素材は外部ツールで用意)



素材は 画像生成 → Tripo AI → Mixamo → Blender の流れで用意し、実装に時間を使う体制にした

コードの構成 — エンジン層とゲーム層

— エンジン層 `src/engine/` 75ファイル・13,174行

フォルダ	役割	数	行数	
engine/asset	アセット管理・VFS・バイク(pak / DDS / PCM / MDL)	13	1,539	■
engine/audio	XAudio2 と X3DAudio による再生・3D 音響	3	1,546	■
engine/component	GameObject と Component の土台	3	128	
engine/core	シーンとオブジェクトの管理・時間	8	773	■
engine/debug	デバッグ描画・計測(Profiler)	3	214	
engine/feedback	ヒットストップ・カメラ演出・画面効果	10	620	■
engine/graphics	DirectX 11 描画・glTF・カメラ・文字描画	16	4,606	■
engine/input	キーボード・マウス・ゲームパッド	10	2,284	■
engine/math	数学(ベクトル)	1	114	
engine/physics	衝突判定(コライダー・レイヤー)	8	1,350	■

合計 155 ファイル・24,938 行(.cpp / .h、コメント・空行を含む)。ほかに HLSL シェーダー29本。外部ライブラリは external/ に分けて置いている。「エンジン層」は他のゲームでも使える部品、「ゲーム層」はこの作品専用のコード、という切り分け。

— ゲーム層 `src/game/` 80ファイル・11,764行

フォルダ	役割	数	行数	
game/actors	プレイヤー・敵・弾・斬撃	14	3,505	■
game/app	ゲーム進行・チュートリアルほか	9	1,162	■
game/effects	爆発・虚装(触手)・パーティクルなど	21	3,837	■
game/ui	HP バー・スコア・ウェーブ表示・ポーズ	16	1,518	■
game/weapon	武器(剣・虚装)	5	854	■
game/world	地形・街・木・草・空	15	888	■

● 単独 個人開発(チームなし)。入力は `input_manager` と `game_controller` を私が実装した。

アセットの追加を「1行」で済ませる — X-Macro と AssetManager

— 課題

テクスチャやモデルを1つ足すたびに、enum・パスの配列・キャッシュの3か所を同じ順番で直す必要がある。順番がずれると、別のアセットが読み込まれる。

— やったこと

定義を `asset_def.h` の1か所に集約した①。TEXTURE(ID, パス) を1行書くだけで、マクロを差し替えて展開し、enum②とパスの配列③を同じリストから作る(X-Macro)。

AssetManager は ID から取得するだけの窓口。一度作ったテクスチャはキャッシュし、起動時に `PreloadAll` でまとめて読む。

— ポイント

- Release ではバイク済みの `.dds` を読む(デコードもミップ生成も無し)④。Debug は生の png / hdr を読み、その場でミップを作る。
- 1行の追加で済むので、アセットの追加時にバグが入り込まない。

```
asset_def.h
15 #define TEXTURE_LIST \
16     TEXTURE(FIELD, "assets/textures/environment/field.png") \
17     TEXTURE(FIELD_NORMAL, "assets/textures/environment/field_normal.png") \
18     :
32 #define TEXTURE(id, path) id,
33 enum class TextureID {
34     TEXTURE_LIST
35     MAX
36 };

asset_def.h L15-L17 / L32-L36

asset_manager.cpp
21 #define TEXTURE(id, path) path,
22 static const char* g_texturePaths[static_cast<int>(TextureID::MAX)] = {
23     TEXTURE_LIST
24 };
25 #undef TEXTURE
26 :
95 if (AssetFileSystem::IsPackMode()) {
96     ///--- Release: バイク済み .dds を読む (デコード・ミップ生成なし) ---
97     std::string path = origPath;
98     size_t dot = path.find_last_of('.');
99     if (dot != std::string::npos) path = path.substr(0, dot) + ".dds";
100 :
118 } else {
119     ///--- Debug: 生 png/hdr をランタイムでデコード+ミップ生成 ---
120     std::vector<uint8_t> fileData;
121     if (!AssetFileSystem::ReadFile(origPath, fileData)) return nullptr;
```

asset_manager.cpp L21-L25 / L95-L100 / L118-L121

起動を速くする — 自作のアセットベイクと VFS

— 課題

起動のたびに、MP3・PNG のデコード、ミップ生成、GLB のパースが走り、起動が遅い。

— やったこと

ゲーム起動時、`data.pak` が無い・古いときだけベイクを実行し、1つの pak にまとめる。

- MP3 はPCM にデコードしてから格納する③。
- PNG / HDR はミップ付きの DDS にする④(非 HDR・非法線は BC7 圧縮)。
- GLB はベイク済みの `.mdl` にする。頂点・ボーン・アニメーションのキーを、GPU へコピーするだけの形へ事前変換しておく(ワインディング反転などもここで焼き込む)。テクスチャは BC7。

— ポイント

- 作り直すかは、pak のフォーマット版数①と、ソースの更新時刻②で判定する。
- pak の先頭は、作っている間は `"PAK?"` にしておき、書き終えてから `"PACK"` に戻す⑤。途中で落ちて壊れた pak を読ませない。

```
asset_baker.cpp
63     if (n != 1 || memcmp(h.magic, "PACK", 4) != 0 || h.version != PackFormat::VERSION) { ①
64         return true;
65     }
66     ...
72     auto pakTime = fs::last_write_time(pakPath);
73     // srcDir 内を再帰スキャンして、pak より新しいファイルがあるかチェック
74     for (auto& entry : fs::recursive_directory_iterator(srcDir)) {
75         if (!entry.is_regular_file()) continue;
76         if (entry.last_write_time() > pakTime) return true; ②
77     }
78     ...
581     memcpy(header.magic, "PAK?", 4); // 途中で落ちて壊れた pak を読ませない。完了時に "PACK" へ書き換え⑤
582     ...
610     if (ext == ".mp3" || ext == ".MP3") { ③
611         bool is3d = Is3DAudioPath(relPath); // ← .pcm に変える前の .mp3 パスで判定
612         if (!DecodeMP3ToPCM(srcPath, fileData, is3d)) {
613             ...
623         } else if (ext == ".png" || ext == ".PNG" ||
624                 ext == ".hdr" || ext == ".HDR" ||
625                 ext == ".jpg" || ext == ".JPG") {
626             if (!EncodeImageToDDS(srcPath, fileData)) { ④
```

asset_baker.cpp L63-L65 / L72-L78 / L581 / L610-L612 / L623-L626

速くなったかを数字で確かめる — 起動時間の計測

課題

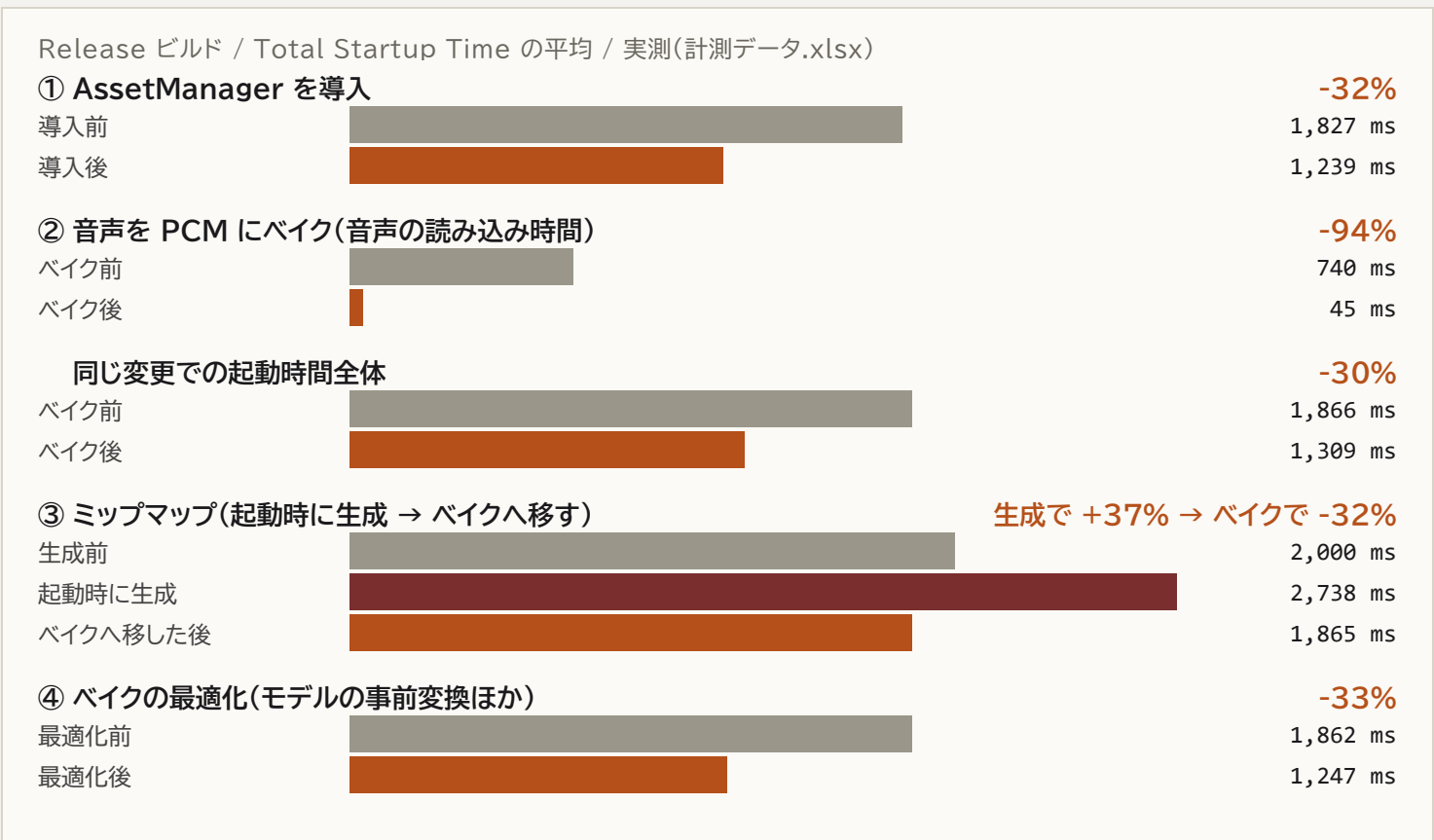
「速くなった気がする」で終わらず、変更のたびに起動時間を計測して、効いたかどうかを数字で確かめたかった。

やったこと

各処理に `ScopedTimer` を入れ、起動の内訳（アセット読み込み・音声・シーン初期化）を毎回ログに出す。変更の前後で、Release ビルドを同じ条件で3回前後測って平均した。

わかったこと

- 音声のデコードが、起動の中でも重かった（約740ms）。
- ミップマップを起動時に作ると、逆に遅くなった。生成をバイクへ移すと、元の水準まで戻った。



衝突のルールをゲームから切り離す — レイヤーと関数表

課題

衝突判定のルールを、敵・弾・剣・地形などのゲームのロジックから切り離したい。「この敵はあの弾と当たらない」を、各クラスの中の if で書くと散らかる。

やったこと

Unity の Physics Layer Matrix と同じ発想にした。GameObject は自分のレイヤーを1つ持つだけ。衝突するかどうかは、PhysicsWorld が持つレイヤー×レイヤーの2次元 bool 配列で決める②③。

初期値は「全部当たる」にして、例外だけを書く。設定は対称になるよう、片方を書けば両方に効く。

形の組み合わせ(球・箱・OBB・カプセル)は、4×4の関数表で振り分ける①。

ポイント

- 静的なレイヤー(地形・草木)は動かさず、動くほうだけを押し戻す。無駄な処理をしない。
- OBB×OBB は使う場面がないため未実装 (DispNotImpl)。必要になった時に足す方針。

```
physics_world.cpp

559 IntersectFn s_Table[kTypeCount][kTypeCount] = {
560     //          b=Sphere,   b=Box,      b=OBB,      b=Capsule
561     /* a=Sphere */ { DispSS,   DispSB,    DispSO,    DispSC },
562     /* a=Box     */ { DispBS,   DispBB,    DispBO,    DispBC },
563     /* a=OBB     */ { DispOS,   DispOB,    DispNotImpl, DispOC },
564     /* a=Capsule */ { DispCS,   DispCB,    DispCO,    DispCC },
565 };

604 // PlayerAttack は Enemy とだけ衝突すればよい
605 SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Player, false); // Player本体には当たらない
606 SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::PlayerAttack, false); // 複数剣同士も無効
607 SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Terrain, false); // 地形は貫通
608 SetLayerCollision(CollisionLayer::PlayerAttack, CollisionLayer::Foliage, false); // 草木も貫通

616 // Foliage (草木) : Enemyとは絡まない (貫通させる)
617 SetLayerCollision(CollisionLayer::Foliage, CollisionLayer::Enemy, false);
618 SetLayerCollision(CollisionLayer::Foliage, CollisionLayer::EnemyBullet, false);

632 bool PhysicsWorld::CanCollide(Collider* a, Collider* b) const {
633     return m_LayerMatrix[static_cast<int>(a->GetLayer())][static_cast<int>(b->GetLayer())];
634 }
```

physics_world.cpp L559-L565 / L604-L608 / L616-L618 / L632-L634

斜めの剣を判定する — OBB を「ローカル空間の AABB」に帰着させる

— 課題

剣の当たり判定は斜めに構える。AABB(軸に揃った箱)では、振りの向きに合わない余白が出て、当たっていないのに当たる。

— やったこと

向きを持てる **OBB** を入れた。相手の球やカプセルを、**OBB** のローカル座標へ移して①しまえば、OBB は「ただの AABB」になる。あとは、最近接点を箱の範囲へクランプするだけ②。

カプセルは、まず芯の線分上で箱の中心に近い点を取り③、箱へクランプし、もう一度線分へ射影する(2回の近似)。

結果は、ローカルの法線をワールドへ戻して返す。

— ポイント

- OBB×球 は「ローカルへ移して AABB×球 に帰着」させるだけなので、新しい判定式を作らずに済んだ。
- 空間分割は入れていない。総当たりで足りている間は増やさず、必要になったら実測してから入れる方針(YAGNI)。

```
physics_world.cpp
185 float localX = d.x * ax.x + d.y * ax.y + d.z * ax.z;
186 float localY = d.x * ay.x + d.y * ay.y + d.z * ay.z;
187 float localZ = d.x * az.x + d.y * az.y + d.z * az.z;
188
189 // OBBのハーフサイズにクランプ (最近接点をローカル空間で計算)
190 Vector3 hs = obb->GetHalfSize();
191 float cx = std::max(-hs.x, std::min(localX, hs.x));
192 float cy = std::max(-hs.y, std::min(localY, hs.y));
193 float cz = std::max(-hs.z, std::min(localZ, hs.z));
...
352 Vector3 a1 = toLocal(cap->GetPointA());
353 Vector3 a2 = toLocal(cap->GetPointB());
354
355 // capsule core closest to the box center, then clamp to the box, then
356 // re-project onto the core (same 2-pass refinement as CapsuleVsBox)
357 Vector3 segPt = ClosestPointOnSegment(Vector3{ 0.0f, 0.0f, 0.0f }, a1, a2);
358 float cx = std::max(-hs.x, std::min(segPt.x, hs.x));
359 float cy = std::max(-hs.y, std::min(segPt.y, hs.y));
360 float cz = std::max(-hs.z, std::min(segPt.z, hs.z));
361 Vector3 boxPt = { cx, cy, cz };
362 segPt = ClosestPointOnSegment(boxPt, a1, a2);
```

physics_world.cpp L185-L193 / L352-L362

ボスが天高く飛んだ — 自分自身との衝突

— 起きたこと

ボスの当たり判定を実寸に合わせて、1つの **GameObject** に2つのコライダー(下半身の Box と頭の Sphere)を持たせた。すると、ボスが天高く飛んでいった。

— 原因

物理の判定は、GameObject のペアではなくコライダーのペアで総当たりしていた。ボスの Box と Sphere は、設計上めり込んだ配置になるので、同じボスのコライダー同士が衝突と判定された。

押し戻しは「両方動く場合は半分ずつ」。同じオブジェクトに2回かかって互いに上書きし合い、位置が毎フレーム上へずれ続けた。

— 直したこと

ペアの先頭で、同じ **GameObject** 同士は飛ばす^①。1オブジェクト1コライダーを前提にしていた見落としだった。

— なぜ2つに分けたか

下から突進されるとボスが浮いた。単一のカプセルは下端が丸く、低い位置への接触で押し戻しが斜め上に向いたため。下半身を平らな Box にして解消した。

enemy.cpp

```
104 auto* body = AddComponent<BoxCollider>();
105 body->SetSize({ bodyHalfWidth * 2.0f, bodyHeight, bodyHalfWidth * 2.0f });
106 body->SetOffset({ 0.0f, bodyHeight * 0.5f, 0.0f });
107 body->SetTag(CollisionTag::Enemy);
108 body->SetLayer(CollisionLayer::Enemy);
109
110 auto* head = AddComponent<SphereCollider>();
111 head->SetRadius(headRadius);
112 head->SetOffset({ 0.0f, bodyHeight, 0.0f }); // 肩(Boxの天井)に乗せる
113 head->SetTag(CollisionTag::Enemy);
```

enemy.cpp L104-L113

physics_world.cpp

```
659 GameObject* objB = colB->GetGameObject();
660 if (!objB || objB->IsDestroyed()) continue;
661
662 // 複数のColliderを持つGameObject同士で、同じGameObject同士の衝突は無視する
663 if (objA == objB) continue;
664
665 // Layerフィルタ
666 if (!CanCollide(colA, colB)) continue;
```

physics_world.cpp L659-L666

動きの繋がりを1つの表にする — キャンセル表と先行入力

課題

アクションの手触りは、「どの動きから、どの動きへ、いつから移れるか」で決まる。条件を if で散らすと、調整のたびにコード中を探し回ることになる。

やったこと

すべてを1つの表に集約した①。値は「その行動へ移れるようになるまでの秒数」。調整はこの表だけを見ればよい。

今の状態	ジャンプ	回避	攻撃	強攻撃
回避中	0.10	0.25	0.10	0.10
攻撃中	0.15	0.05	0.00	0.15
被弾中	—	0.20	—	—
通常・走り	0.00	0.00	0.00	0.00

その行動へ移れるまでの秒数(player.cpp L462-L489 の値)。— は移れない。

状態の変更は SetState だけを通し②、経過時間のリセット漏れを防ぐ。移れる時間より早く押された入力は0.18秒だけ保持し、受付が開いた瞬間に出す③。

```

player.cpp

451 void Player::SetState(PlayerState s) {
452     if (s != PlayerState::Attack) m_LungeTimer = 0.0f;
453     m_State = s;
454     m_StateTimer = 0.0f;
455 }
456
457 case PlayerState::Dodge:
458     return { 0.10f, 0.25f, 0.10f, 0.10f };
459
460
461 if (acceptTime < 0.0f) return false; // NEVER
462 return m_StateTimer >= acceptTime;
463
464 void Player::BufferAction(ActionKind a) {
465     m_BufferedAction = a;
466     m_BufferTimer = INPUT_BUFFER_TIME;
467 }
468
469 m_BufferTimer -= dt;
470 if (m_BufferTimer <= 0.0f) { // 古くなった入力は捨てる
471     m_BufferedAction = ActionKind::None;
472     return;
473 }
474
475 if (!CanCancelInto(m_BufferedAction)) return; // まだ受付時間外。次フレームで再挑戦
    
```

player.cpp L451-L455 / L476-L477 / L503-L504 / L507-L510 / L520-L526

回避にリスクと見返りを — ジャスト回避

課題

回避の間ずっと無敵だと、連打すれば永久に無敵になる。

回避にリスクも見返りもなくなってしまう。

やったこと

無敵は、回避の出だしの0.35秒だけにした①（回避全体は0.5秒）。その間に当たったら「ジャスト回避」とする②。

成功すると、反撃の猶予が1.2秒できて与ダメージが3倍（通常は1）になる③。同時に、時間が0.45秒だけ0.25倍になり、画面の白フラッシュと専用の演出を出す。

ポイント

- 判定を先読みしない。TakeDamage に届いた時点で「本当に当たっていた」ことが保証されているので、その瞬間に無敵時間中かを見るだけでよい。
- スローは、ヒットストップと同じ仕組み（時間の進み方を変える）を使い回している。入力の実時間なので、自分だけ速く動ける感覚になる。

player.h

```
131 static constexpr float DODGE_IFRAME = 0.35f; // 回避の無敵時間（回避開始から）
132 static constexpr float COUNTER_TIME = 1.20f; // ジャスト回避後の反撃猶予
133 static constexpr int COUNTER_DAMAGE = 3; // 反撃の与ダメージ（通常は1）
134 static constexpr float SLOWMO_TIME = 0.45f; // スロー演出の長さ
135 static constexpr float SLOWMO_SCALE = 0.25f; // スロー中の時間の進み方
...
389 bool IsDodgeInvincible() const {
390     return m_State == PlayerState::Dodge && m_StateTimer <= DODGE_IFRAME;
391 }
```

player.h L131-L135 / L389-L391

player.cpp

```
992 if (IsDodgeInvincible()) {
993     OnPerfectDodge();
994     return;
995 }
...
1174 m_CounterTimer = COUNTER_TIME;
1175
1176 // 世界だけ遅くする。プレイヤーの入力は実時間なので「自分だけ速い」感覚になる
1177 HitStop::Trigger(SLOWMO_TIME, SLOWMO_SCALE);
```

player.cpp L992-L995 / L1174-L1177

避けられる攻撃にする — 予兆と当たり判定

— 課題

敵の攻撃に予告が無いと、ジャスト回避が運になる。かといって、画面が光りすぎても遊びの邪魔になる。

— やったこと

攻撃の溜めの間だけ、頭上のグローと体の色で予告する。時間に応じて大きくし①、終盤ほど点滅を速める②。色は攻撃の種類ごとに変えた(近接=橙、射撃=赤、ボスの叩きつけ=黄)。

体の色は、元の色と警告色を混ぜる(ピークで約5割)③。発射の直前(残り20%)には、白いフラッシュで「今」を知らせる④。

— 攻撃の当たり判定

近接の攻撃は、前方に箱の当たり判定を生成する⑤。以前は水平の距離だけで判定していて、ビルの上のプレイヤーにも当たっていた。箱にしたので、高さも見る。

enemy.cpp

```
200 float t = m_StateTimer / startup; // 0→1
201 float grow = 0.25f + 0.75f * (t * t); // 出だしから見えるように下駄をはかせる ①
...
204 float blink = 1.0f;
205 if (t > 0.45f) {
206     float f = (t - 0.45f) / 0.55f;
207     blink = 0.5f + 0.5f * sinf(m_StateTimer * (26.0f + 55.0f * f)); ②
208 }
...
240 float mix = (0.10f + 0.40f * grow) * (0.65f + 0.35f * blink);
241 model->SetTint(true, { cr, cg, cb, 1.0f }, mix); ③
...
246 if (t > 0.80f) { ④
247     float s = (t - 0.80f) / 0.20f; // 0→1
248     float cs = TELEGRAPH_CORE_MAX * scale * (0.35f + 0.65f * s);
249     m_TelegraphCore->SetSize(cs, cs);
250     float w = 2.0f + 5.0f * s; // スナップも控えめ
251     m_TelegraphCore->SetColor({ w, w, w, 1.0f });
```

enemy.cpp L200-L201 / L204-L208 / L240-L241 / L246-L251

enemy_attack.cpp

```
13 auto* col = AddComponent<BoxCollider>(); ⑤
14 col->SetSize(m_BoxSize);
15 col->SetOffset({ 0.0f, 0.0f, 0.0f });
16 col->SetLayer(CollisionLayer::EnemyBullet); // 敵の攻撃用レイヤー(Enemy とは当たらない設定済み)
17 col->SetTrigger(true); // 押し戻さない。当たり判定だけ
...
38 if (Player* player = dynamic_cast<Player*>(info.other)) {
39     m_Hit = true;
40     player->TakeDamage(m_Damage, m_KnockDir);
41 }
```

enemy_attack.cpp L13-L17 / L38-L41

モデルを使わず、曲線から武器を組み立てる — 虚装

— 課題

「体から伸びる影の武器(虚装)」は形が決まっていない。固定のメッシュだと、動きの表現に合わない。

— やったこと

モデルを持たず、毎フレーム頂点を組み立てて描く。振り・溜め・突きは、2次ベジエ曲線の制御点を動かすだけで表す①。
物理シミュレーションは持たない。

曲線の各点に断面の座標系を運び②、平たい楕円の断面を掃引してチューブにする。カメラ正対の板にすると、どこから見ても厚みが無くなるため。

— ポイント

- 長さ方向に等間隔の瘤を立てて節を作る③。
- 周期の違う正弦を掛け合わせた痙攣を重ねる④。単純な正弦1本だと、きれいに振動しているだけに見えるため。
- シェーダーはライティング無し(UNLIT)なので、陰影を頂点カラーに焼き込む(ランバート・リム・縁)。

```
phantom_arm.cpp

31 static Vector3 Bezier(const Vector3& p0, const Vector3& p1, const Vector3& p2, float t) {
32     const float u = 1.0f - t;
33     return p0 * (u * u) + p1 * (2.0f * u * t) + p2 * (t * t);
34 }
35
360 // 前の u から接線成分を抜いて、直交させ直す
361 U = carriedU - tan * Vector3::dot(carriedU, tan);
362
363
364 float barb = sinf(u * BARB_FREQ + phase * 1.7f);
365 barb = (barb > 0.0f) ? powf(barb, BARB_SHARP) : 0.0f;
366 const float swell = 1.0f + BARB_AMP * barb;
367
368 const float hw = pt.halfWidth * swell; // 幅 (刃の広い方)
369 const float ht = pt.halfWidth * TUBE_THICK * swell; // 厚み (刃の薄い方)
370
371 const float twEnv = powf(u, 0.70f) * m_Deploy;
372 const float twitch = sinf(u * TWITCH_FREQ + m_Time * TWITCH_SPEED + phase * 3.1f)
373     * sinf(m_Time * TWITCH_BEAT + phase)
374     * TWITCH_AMP * twEnv;
375
376 const Vector3 center = pt.pos + U * twitch + V * (twitch * 0.6f);
```

phantom_arm.cpp L31-L34 / L330-L331 / L349-L354 / L362-L367

手応えを組み合わせで作る — ヒットストップと被弾演出

— 課題

当たった手応えは、ダメージ数字ではなく、画面と音と振動で決まる。1つの演出だけ強くしても、まとまった手応えにならない。

— やったこと

ヒットストップは、時間の進み方(`TimeScale`)を一瞬だけ下げて作る①。カウントダウンは

`UnscaledDeltaTime` で行うので、止まっている間も戻る時刻がずれない。重なったときは、長いほうを採る②。

被弾のときは、ヒットストップ・カメラの揺れ・パッドの振動・カメラごと押される動き・画面の効果・音を、同じ入口から同時に出す③。斬撃・ジャスト回避・覚醒でも、同じ部品を強さだけ変えて使う。

— ポイント

- 演出は「エンジン層」の部品(`engine/feedback`)。ゲーム側は強さと長さを渡すだけ。

```
hit_stop.cpp

17 void HitStop::Trigger(float duration, float scale) {
18     if (s_Timer <= 0.0f) {
19         // 新規発動：現在のTimeScaleを保存
20         s_OriginalScale = Time::GetTimeScale();
21     }
22     // 既存のヒットストップより長ければ上書き（重なった時の対応）
23     if (duration > s_Timer) {
24         s_Timer = duration;
25     }
26     Time::SetTimeScale(scale);
27 }
...
33 s_Timer -= Time::GetUnscaledDeltaTime();
```

hit_stop.cpp L17-L27 / L33

```
player.cpp

1020 HitStop::Trigger(0.15f, 0.05f);
1021 CameraShake::Trigger(1.2f, 0.42f);
1022 InputManager::StartVibration(0.8f, 0.32f); // 画面の揺れと同じ強さ・長さで振動させる
1023 CameraPush::Trigger(knockbackDir, 0.7f, 0.30f); // 殴られた方向へカメラごと押される
1024 ScreenFX::TriggerDamage(1.0f, 0.95f);
1025 ScreenFX::TriggerFlash(0.18f, 0.09f); // 一瞬白く抜いて食らった瞬間を強く
1026 Audio_PlaySE3D(AudioID::SE_DAMAGE, m_Position.x, m_Position.y, m_Position.z);
```

player.cpp L1020-L1026

担当のまとめと、次の課題

155

ソースファイル数

— 担当の要点

- 個人開発として、エンジン(描画・物理・アセット・音)からゲームの中身まで、すべて設計・実装した。
- 「追加・調整が1か所で済む形」を優先した(X-Macro、レイヤーの表、キャンセル表)。
- 速度の話は計測してから手を入れ、効かなかったものも記録に残した。

24,938

ソース行数(.cpp / .h)

29

HLSL シェーダー

— 次に手を入れたいところ

- 衝突判定の総当たり: 現状は全コライダーの組み合わせを調べている(physics_world.cpp L647~)。敵の数を増やすなら、空間分割を実測とセットで入れる。
- OBB×OBB: 未実装のまま。武器同士の判定など、必要になった時に足す。
- ファイルの大きさ: `player.cpp` と `phantom_arm.cpp` は各1,300行超。状態の管理と演出を分けたい。

約1.25秒

Release の起動時間(以前は約1.86秒)